

EECS3311 Software Design

Fall 2026

Professor: Song Wang

Course Project: Design, Build, and Test an AI Agent

Orion Threat Intelligence

Name: Saransh Mahajan

Student ID: 221563945

Project Overview

Security Operations Centers (SOCs) and Cyber Threat Intelligence (CTI) teams suffer from severe alert fatigue and data fragmentation. Analysts must manually ingest hundreds of vulnerability advisories published across disparate catalogs (such as the National Vulnerability Database [NVD] and CISA Known Exploited Vulnerabilities [KEV]), parse lengthy technical bulletins, and cross-reference those external threat indicators with internal endpoint audit logs. Orion Threat Intelligence automates this pipeline, bridging the gap between external threat intelligence and internal risk correlation to drastically reduce triage time and mitigate human error.

Target Users:

1. **SOC Analysts:** Personnel who need rapid summarization of dense vulnerability disclosures and automated extraction of actionable Indicators of Compromise (IoCs).
2. **Threat Intelligence Specialists:** Analysts tracking emerging adversary Tactics, Techniques, and Procedures (TTPs) across external feeds.
3. **Security Supervisors:** Leaders requiring automated risk triage, security control gap analyses, and exportable executive threat briefings.

Agent Description:

The Orion Threat Intelligence AI agent acts as an autonomous analytical assistant. It can ingest live external threat feeds, summarize highly technical vulnerability bulletins, and answer natural-language queries about the current threat landscape. Furthermore, it extracts structured network indicators (IPs, domains, hashes) from unstructured text, correlates internal access logs against known external attack patterns, and dynamically generates actionable mitigation playbooks for specific threats.

While traditional deterministic software excels at fetching API payloads and running SQL queries, it fails when processing ambiguous data and to contextualize data towards a specific environment. Deterministic code cannot easily synthesize a five-page unstructured threat report, infer semantic overlaps between a new ransomware variant and internal access anomalies, or formulate a customized mitigation plan based on specific organizational infrastructure. An AI agent is uniquely equipped to handle this non-deterministic reasoning, planning, and contextual synthesis while relying on deterministic tools to pull the raw data.

AI/LLM Models to Scope

1. **Google Gemini 2.5 Flash:** Serving as the primary reasoning engine, utilized for its massive 1-million token context window, which is necessary for ingesting lengthy threat advisories and audit logs simultaneously.
2. **Groq LLaMA 3.3 (70B):** Serving as a secondary/fallback model, utilized for high-speed, low-latency formatting tasks like JSON extraction and IoC normalization.

Architecture

The AI models are strictly isolated from direct system execution to ensure human-in-the-loop safety. The deterministic Java backend (**AgentController**) coordinates the workflow. When a task requires reasoning, an internal **PromptBuilder** retrieves factual data from the local SQLite database and formats a structured prompt. This payload is sent asynchronously to the LLM via a REST HTTP client (**LLMClient**). The AI model returns a structured JSON response (e.g., a mitigation plan or risk score), which the deterministic software parses, validates, and either displays in the JavaFX GUI or queues for execution via a **CommandManager**.

Feature Specification

F01 - External Threat Advisory Ingestion

- **Description:** Fetches, parses, and stores external threat advisories from vulnerability catalogs (e.g., NVD, CISA KEV) into the local SQLite database.
- **User Interaction:**
 - **GUI:** The user clicks the "Sync External Feeds" button on the Threat Dashboard.
 - **CLI:** The user executes `orion-cli feeds sync --source=all`.
- **Input:** Feed source selection and synchronization parameters (e.g., date limits).
- **Output:** An updated local database and an ingestion status summary showing records added.
- **AI Involvement:** Deterministic.

- **Expected Workflow:** The system triggers an HTTP GET request to the selected remote API, parses the incoming JSON/XML response into Java domain objects, and saves them directly to the local SQLite repository.
- **Error/Alternative Cases:** If a network timeout or API rate limit occurs, the system catches the exception, leaves the existing local database untouched, and notifies the user with a connection error message.

F02 - Automated Threat Bulletin Summarization

- **Description:** Analyzes lengthy technical vulnerability advisories to generate concise executive summaries highlighting the attack vector, affected systems, and operational severity.
- **User Interaction GUI:** The user selects an advisory from the data table and clicks "Generate Summary".
- **Input:** The raw text of a threat advisory or a specific CVE ID.
- **Output:** A structured summary card detailing the executive impact and exploited mechanisms.
- **AI Involvement:** AI-based.
- **Expected Workflow:** The agent retrieves the advisory text from the local database, frames it within a summarization prompt, and sends it to the LLM. The LLM returns a concise summary, which the system renders in the GUI.
- **Error/Alternative Cases:** If the LLM service is unavailable, the system displays a deterministic fallback summary using regex-extracted CVSS metrics and base severity ratings pulled from the raw payload.

F03 - Natural Language Threat Landscape Querying

- **Description:** Allows analysts to ask ad-hoc questions about stored vulnerabilities (e.g., *"What active threats target Windows servers?"*).
- **User Interaction:**
 - **GUI:** The user types a query into the chat interface.
 - **CLI:** The user runs orion-cli query "[question]".

- **Input:** A natural language query string.
- **Output:** A synthesized, contextual intelligence response citing relevant CVEs.
- **AI Involvement:** Hybrid.
- **Expected Workflow:** The agent determines the query intent, deterministically searches the local SQLite repository for matching advisories, and passes the retrieved results alongside the user's query to the LLM. The LLM synthesizes a grounded, conversational answer.
- **Error/Alternative Cases:** If no local database records match the query, the agent informs the user that local intelligence is lacking rather than hallucinating an answer and suggests running a feed sync.

F04 - Insider Risk Log Correlation

- **Description:** Correlates internal system access logs with external threat signatures to detect potential insider risks or compromised accounts.
- **User Interaction:**
 - **GUI:** The user uploads an audit log (CSV/JSON) and clicks "Correlate".
 - **CLI:** The user runs `orion-cli correlate --file=[path]`.
- **Input:** A local CSV or JSON file containing internal endpoint or access logs.
- **Output:** A prioritized list of risk alerts indicating anomalous behaviors mapped to known threat TTPs.
- **AI Involvement:** Hybrid.
- **Expected Workflow:** The system deterministically parses the uploaded log file to extract anomalous events (e.g., bulk file access). The agent then prompts the LLM with these anomalies alongside recent high-severity threat profiles. The LLM identifies overlaps, returning structured risk alerts to the UI.
- **Error/Alternative Cases:** If the uploaded file is malformed or uses unsupported headers, the deterministic parser immediately rejects it and displays a file formatting error without invoking the LLM.

F05 - Automated Mitigation Strategy Generation

- **Description:** Creates step-by-step remediation playbooks (e.g., firewall block rules, configuration changes) tailored to a specific threat.
- **User Interaction:**
 - **GUI:** The user clicks "Generate Mitigation" on a threat profile card.
- **Input:** A Threat Advisory ID and the target enterprise infrastructure profile.
- **Output:** A structured mitigation plan containing executable security commands.
- **AI Involvement:** AI-based.
- **Expected Workflow:** The agent retrieves the threat details and local infrastructure configuration, constructs a prompt, and queries the LLM. The LLM generates a mitigation playbook, which the system parses into deterministic, executable command objects for user review.
- **Error/Alternative Cases:** If the LLM generates an unrecognized or highly destructive command format, the system flags the plan as "Requires Manual Review" and disables the automated execution buttons.

F06 - Structured Indicator of Compromise (IoC) Extraction

- **Description:** Scans unstructured threat intelligence text to extract, normalize, and categorize IoCs (IP addresses, domains, file hashes).
- **User Interaction:**
 - **GUI:** The user pastes unstructured text into the extraction panel and clicks "Extract".
 - **CLI:** The user pipes a text file into `orion-cli ioc extract --file=[path]`.
- **Input:** Unstructured threat text or a file path.
- **Output:** A deduplicated, structured table of validated IoC records.

- **AI Involvement:** Hybrid.
- **Expected Workflow:** The system first applies deterministic regular expressions to extract standard IPs and hashes. It then sends complex or obfuscated segments to the LLM to normalize (e.g., de-fanging URLs). Finally, all indicators are deduplicated and displayed.
- **Error/Alternative Cases:** If no valid indicators are found in either the deterministic or LLM extraction phases, the system returns an empty table and notifies the user that no recognizable IoCs were present.

F07 - Executive Threat Intelligence Report Export

- **Description:** Compiles selected threat summaries, correlated alerts, and mitigation playbooks into a formatted Markdown or PDF briefing document.
- **User Interaction:**
 - **GUI:** The user selects items from a table and clicks "Export Report".
 - **CLI:** The user runs `orion-cli report export --ids=[ID-List] --format=md`.
- **Input:** A list of threat IDs and the desired output format/path.
- **Output:** A compiled briefing document saved to the local filesystem.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** The system queries the local repository for the specified records, injects the data into a predefined report template, and writes the final document to the user's hard drive using Java File I/O.
- **Error/Alternative Cases:** If the system encounters insufficient disk write permissions or an invalid destination path, it catches the I/O exception and prompts the user to select a different directory.

F08 - Historical Threat Repository Search

- **Description:** Performs fast, deterministic multi-criteria filtering across cached vulnerabilities without consuming LLM API limits.

- **User Interaction:**

- **GUI:** The user sets filters (date range, severity, vendor) in the search panel and clicks "Search".
 - **CLI:** The user runs `orion-cli repo search --min-cvss=7.0`.
- **Input:** Search filters including keywords, numeric CVSS thresholds, and date ranges.
- **Output:** A filtered list of matching threat advisory records.
- **AI Involvement:** Deterministic.
- **Expected Workflow:** The system maps the user's GUI/CLI inputs to a parameterized SQL query, executes it against the local SQLite database, and returns the resulting records to populate the application's data table.
- **Error/Alternative Cases:** If the user inputs invalid filter parameters (e.g., a CVSS score > 10.0), the presentation layer catches the error deterministically and displays an inline validation message.

F09 - Dynamic Advisory Triage & Priority Scoring

- **Description:** Automatically evaluates a queue of new threat bulletins against local organizational infrastructure to assign customized priority scores (CRITICAL, HIGH, MEDIUM, LOW).
- **User Interaction:**
 - **GUI:** The user navigates to the Alert Inbox and clicks "Auto-Triage".
 - **CLI:** The user runs `orion-cli triage evaluate-all`.
- **Input:** A list of unranked threat advisories and the local asset configuration.
- **Output:** A prioritized queue of threats with updated severity badges and AI-generated justification notes.
- **AI Involvement:** AI-based.
- **Expected Workflow:** The system fetches untriaged advisories from the database. The agent provides the advisory metadata and the user's tech stack to the LLM. The LLM evaluates the exploitability context, returning a calibrated triage score and rationale.

- **Error/Alternative Cases:** If the LLM rate limit is reached during bulk triage, the system gracefully falls back to sorting by the deterministic base CVSS score provided in the raw feed, flagging those items as "Fallback Triage".

F10 - Security Control Gap Analysis

- **Description:** Analyzes current enterprise defensive controls against a newly identified threat to highlight coverage gaps and vulnerabilities.
- **User Interaction:**
 - **GUI:** The user views an advisory and clicks "Evaluate Defensive Gaps".
 - **CLI:** The user runs `orion-cli audit gap-analysis --cve=[CVE-ID]`.
- **Input:** A target Threat Advisory ID and the organization's active security control inventory.
- **Output:** A structured gap matrix listing unmitigated attack vectors and recommended defensive actions.
- **AI Involvement:** AI-based.
- **Expected Workflow:** The system retrieves the threat profile and the active defensive rules from the local database. It formulates a comparative prompt, which the LLM uses to identify missing protections. The resulting gap analysis is presented to the user.
- **Error/Alternative Cases:** If the local security control inventory is empty or unconfigured, the system halts the analysis and alerts the user to define their baseline defenses before proceeding.

EECS 3311 Project

UML Class Diagram

Stage 1

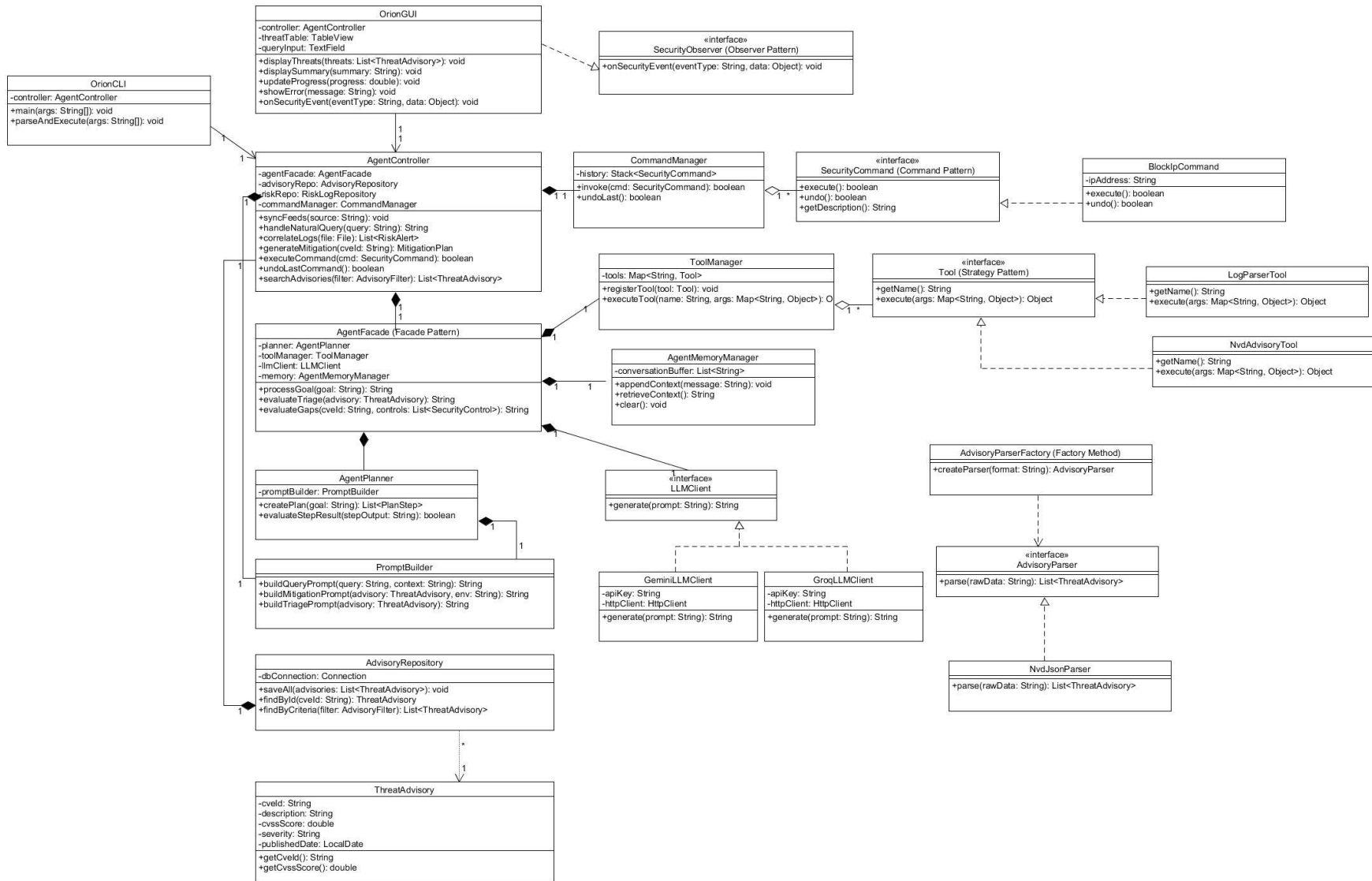


Figure 1: UML Class Diagram

The class diagram for the Orion Threat Intelligence system is structured around a strict architectural separation between traditional deterministic software components and non-deterministic AI agent capabilities. At the boundary layer, the system supports both graphical and command-line interactions through OrionGUI and OrionCLI. These interfaces maintain a unidirectional association with the AgentController, which serves as the central deterministic orchestrator. The AgentController is responsible for managing application state and data persistence, composing the AdvisoryRepository and CommandManager to handle local SQLite database connections and security command tracking, respectively. The traditional domain entities, such as ThreatAdvisory, encapsulate vital structural attributes including the cveld (String), cvssScore (double), and publishedDate (LocalDate).

To cleanly integrate the AI components without polluting the deterministic business logic, the entire reasoning engine is isolated behind the AgentFacade. This subsystem houses the core intelligence of the application. The AgentFacade composes the AgentPlanner, which utilizes a PromptBuilder to structure tasks, and the AgentMemoryManager, which maintains a conversationBuffer attribute to track session context. External LLM interactions are abstracted through the LLMClient interface, which is realized by concrete implementations like GeminiLLMClient and GroqLLMClient. These clients manage their respective HTTP connections and API keys, completely abstracting the network layer away from the agent's reasoning loops. Multiplicities across the system ensure strict structural integrity; for example, the AgentController composes exactly one AgentFacade (1:1), while the ToolManager aggregates a collection of multiple Tool objects (1:*) to execute varied system tasks.

Design Patterns Used in this diagram:

1. Observer Pattern
2. Facade Pattern
3. Strategy Pattern
4. Command Pattern
5. Factory Method Pattern

Design Patterns Explanations

1. The **Observer Pattern** addresses the design problem of handling asynchronous, long-running background operations—such as threat feed synchronization and LLM inference—without freezing the presentation layer. The participating classes are the SecurityObserver interface, the OrionGUI (Concrete Observer), and the AgentController (Subject). The SecurityObserver establishes an onSecurityEvent()

contract, the OrionGUI implements this contract to update its visual components asynchronously, and the AgentController broadcasts state changes as background threads complete. This pattern is highly appropriate because it enforces a strict separation of concerns, decoupling the domain logic from the graphical user interface. If this pattern were omitted, it would be difficult to run the system in a headless state, as the AgentController would require direct, hardcoded references to JavaFX components, breaking the ability for the OrionCLI to share the same underlying controller.

2. The **Strategy Pattern** is implemented to solve the design problem of executing varied data retrieval and parsing algorithms dynamically at runtime based on the agent's planning decisions. The participating classes include the Tool interface (Strategy), NvdAdvisoryTool and LogParserTool (Concrete Strategies), and the ToolManager (Context). In this architecture, the Tool interface defines a universal execute() contract, the concrete classes encapsulate the unique logic required for specific system tasks, and the ToolManager maintains a registry to dispatch execution requests dynamically. This pattern is appropriate because it adheres to the Open/Closed Principle; new integration capabilities can be added by creating new tools without altering the agent's core logic. If this pattern were not used, it would be significantly more difficult to scale the system, as the AgentPlanner would require fragile, hardcoded conditional statements to invoke specific methods for every external utility.
3. The **Facade Pattern** solves the architectural problem of orchestrating a complex, multi-step AI reasoning subsystem without exposing its intricacies to the boundary layer. The participating classes include the AgentFacade (Facade) and its encapsulated subsystems: AgentPlanner, ToolManager, AgentMemoryManager, and LLMClient. The subsystems handle granular tasks like contextual memory retrieval, prompt construction, and API communication, while the AgentFacade exposes simplified, high-level methods such as processGoal() and evaluateTriage(). This approach is appropriate because it establishes a clean, stable entry point for the presentation layer to interact with the non-deterministic agent capabilities. Without the Facade pattern, maintaining the system would be far more difficult, as both the GUI and CLI controllers would need to duplicate the complex orchestration logic required to manage memory, tools, and LLM prompting simultaneously.
4. The **Command Pattern** addresses the critical security problem of executing AI-generated defensive remediations safely, ensuring actions can be tracked, queued, and reversed. The participating classes include the SecurityCommand interface, the BlockIpCommand (Concrete Command), and the CommandManager (Invoker). The SecurityCommand interface mandates execute() and undo() methods, the concrete commands encapsulate the specific infrastructure state (e.g., target IP addresses), and the CommandManager invokes

these actions while pushing them onto an auditable history stack. This pattern is appropriate because it treats infrastructure modifications as discrete objects, allowing security administrators to enforce human-in-the-loop validation before automated agents apply network rules. If this pattern were not used, it would be exceedingly difficult to recover from erroneous AI suggestions, as remediation steps would be executed as untracked procedural method calls lacking any automated rollback mechanism.

5. The **Factory Method Pattern** is utilized to solve the problem of instantiating the correct deserialization algorithms for heterogeneous external threat feeds, which utilize varying schemas like JSON and XML. The participating classes are the AdvisoryParserFactory (Creator), the AdvisoryParser interface (Product), and the NvdJsonParser (Concrete Product). The AdvisoryParserFactory exposes a createParser() method that evaluates a format string to instantiate and return the appropriate AdvisoryParser implementation, which then executes schema-specific parsing logic. This pattern is appropriate because it centralizes object creation, isolating the feed ingestion tools from the complexities of parser initialization. Without this pattern, it would be much more difficult to integrate future vulnerability catalogs, as the core ingestion tools would contain brittle, hardcoded instantiation logic requiring modification every time a new feed format is in

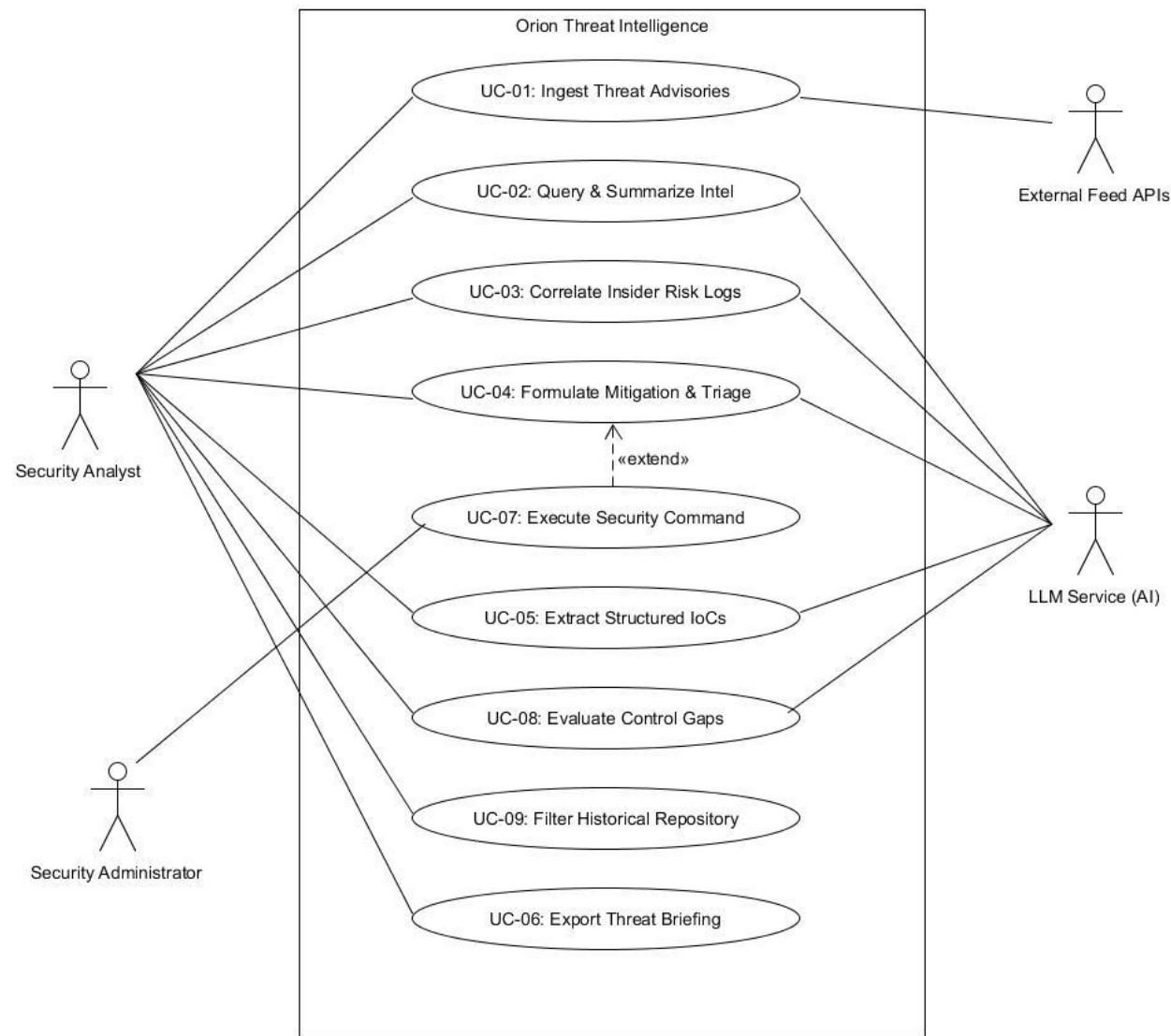


Figure 2: Use-Case Diagram

Detailed Use-Case Descriptions

UC-01: Ingest Threat Advisories

- **Use Case ID:** UC-01
- **Use Case Name:** Ingest Threat Advisories
- **Actor(s):** Security Analyst, External Feed APIs (NVD, CISA)
- **Goal:** Retrieve the latest vulnerability bulletins from external catalogs and store them locally.
- **Preconditions:** Active internet connection; local database is initialized.
- **Trigger:** Analyst clicks "Sync Threat Advisories" or runs the sync CLI command.
- **Main Success Scenario:**
 1. Analyst initiates synchronization for a specified source.
 2. System requests the correct parser from the factory.
 3. The ingestion tool sends an HTTP GET request to the remote endpoint.
 4. Data is parsed and saved to the local SQLite database.
 5. The GUI is updated with a summary of the ingested records.
- **Alternative/Exception Flows:**
 - *Network Timeout:* The system catches an APIException, preserves existing data, and displays an error alert to the user.
- **Postconditions:** The database reflects the most recent external threat intelligence.
- **Related Feature(s):** F01.

UC-02: Query & Summarize Threat Intelligence

- **Use Case ID:** UC-02
- **Use Case Name:** Query & Summarize Threat Intelligence
- **Actor(s):** Security Analyst, LLM Service
- **Goal:** Obtain natural-language answers and executive summaries based on stored vulnerabilities.
- **Preconditions:** The database contains cached threat records; the LLM client is configured.
- **Trigger:** Analyst types a question in the chat bar or requests an advisory summary.
- **Main Success Scenario:**
 1. Analyst submits a query or summary request.
 2. The agent planner identifies the necessary local threat records and retrieves them.
 3. The system combines the retrieved records and conversation history into an analysis prompt.
 4. The LLM service returns a synthesized, markdown-formatted response.
 5. The context memory updates, and the UI displays the response.
- **Alternative/Exception Flows:**
 - *No Local Records Match:* The agent deterministically advises the analyst that local intelligence is lacking rather than hallucinating.
 - *LLM Rate Limit Reached:* The system falls back to a deterministic string parser to display basic CVSS metrics.
- **Postconditions:** The user views the contextual threat data, and memory is updated.
- **Related Feature(s):** F02, F03.

UC-03: Correlate Insider Risk Logs

- **Use Case ID:** UC-03

- **Use Case Name:** Correlate Insider Risk Logs
- **Actor(s):** Security Analyst, LLM Service
- **Goal:** Identify potential insider threats by matching internal access anomalies with external CVE profiles.
- **Preconditions:** Analyst possesses a valid CSV/JSON audit log.
- **Trigger:** Analyst uploads the file and clicks "Correlate".
- **Main Success Scenario:**
 1. Analyst uploads the log file via the UI.
 2. The parser deterministically extracts statistical outliers and access anomalies.
 3. The system prompts the LLM with the anomalies alongside known threat signatures.
 4. The LLM identifies behavioral overlaps and generates structured risk alerts.
 5. Alerts persisted and displayed in the UI.
- **Alternative/Exception Flows:**
 - *Malformed File:* The deterministic parser catches a format violation, aborts the LLM call, and throws a validation error to the UI.
- **Postconditions:** Risk correlations are saved and actionable for the analyst.
- **Related Feature(s):** F04.

UC-04: Formulate Mitigation & Triage Strategy

- **Use Case ID:** UC-04
- **Use Case Name:** Formulate Mitigation & Triage Strategy
- **Actor(s):** Security Analyst, LLM Service

- **Goal:** Automatically assign severity scores to untriaged threats and generate actionable defensive playbooks.
- **Preconditions:** Threat advisories exist in the local database.
- **Trigger:** Analyst clicks "Auto-Triage" or requests a mitigation plan for a specific CVE.
- **Main Success Scenario:**
 1. Analyst initiates triage or mitigation planning.
 2. The system sends the threat profile and local enterprise configuration to the LLM.
 3. The LLM evaluates the exploitability and returns a customized triage score and/or mitigation playbook.
 4. The system parses the response into discrete, executable security commands.
 5. The UI updates the priority badges and displays the playbook.
- **Alternative/Exception Flows:**
 - *LLM Service Down:* The system assigns triage scores based entirely on deterministic CVSS base thresholds found in the raw feed.
- **Postconditions:** Threats are prioritized, and structured plans are linked to their respective records.
- **Related Feature(s):** F05, F09.

UC-05: Extract Structured IoCs

- **Use Case ID:** UC-05
- **Use Case Name:** Extract Structured IoCs
- **Actor(s):** Security Analyst, LLM Service
- **Goal:** Normalize and extract IP addresses, domains, and hashes from unstructured text.
- **Preconditions:** Analyst possesses unstructured threat text.

- **Trigger:** Analyst pastes text and clicks "Extract IoCs".
- **Main Success Scenario:**
 1. Analyst submits unstructured text.
 2. The system executes deterministic regular expressions to find well-formed indicators.
 3. Obfuscated strings are passed to the LLM for de-fanging and normalization.
 4. Results are deduplicated and categorized.
 5. The UI displays the indicators in a structured table.
- **Alternative/Exception Flows:**
 - *No Indicators Found:* The system informs the analyst that no recognizable patterns exist in the text.
- **Postconditions:** Valid IoCs are available for export.
- **Related Feature(s):** F06.

UC-06: Export Threat Briefing

- **Use Case ID:** UC-06
- **Use Case Name:** Export Threat Briefing
- **Actor(s):** Security Analyst
- **Goal:** Compile selected analysis and alerts into a formal document.
- **Preconditions:** At least one record is selected in the UI.
- **Trigger:** Analyst clicks "Export Executive Briefing".
- **Main Success Scenario:**

1. Analyst selects records and chooses a local directory.
2. The system queries the repositories for the entities.
3. Entities are formatted into a markdown document template.
4. The file is written securely to the local disk.

- **Alternative/Exception Flows:**

- *Permission Denied:* The system encounters a write error, halts the export, and prompts the user to select an alternate path.

- **Postconditions:** A completed report is saved to the local file system.

- **Related Feature(s):** F07.

UC-07: Execute Security Remediation Command

- **Use Case ID:** UC-07
- **Use Case Name:** Execute Security Remediation Command
- **Actor(s):** Security Administrator
- **Goal:** Execute or roll back system defense actions generated by the agent.
- **Preconditions:** A mitigation plan containing commands exists.
- **Trigger:** Admin clicks "Execute" or "Undo" on a specific action.
- **Main Success Scenario:**
 1. Admin reviews and approves a specific AI-generated command.
 2. The system executes the operation against the local infrastructure.
 3. The command state is pushed onto the command history stack.

4. The UI reflects the executed status.

- **Alternative/Exception Flows:**

- *Execution Fails:* The command fails to apply at the OS level; the system catches the failure, keeps the undo stack unchanged, and alerts the admin.

- **Postconditions:** Defensive configurations are actively applied or reverted.

- **Related Feature(s):** F05.

UC-08: Evaluate Security Control Gaps

- **Use Case ID:** UC-08

- **Use Case Name:** Evaluate Security Control Gaps

- **Actor(s):** Security Analyst, LLM Service

- **Goal:** Cross-reference known threat requirements against the enterprise's active security controls.

- **Preconditions:** Baseline security controls are populated in the system.

- **Trigger:** Analyst clicks "Evaluate Defensive Gaps" on a threat.

- **Main Success Scenario:**

1. Analyst requests an analysis.
2. The system fetches the active control list and the threat vector details.
3. The LLM evaluates coverage overlaps to identify unmitigated vectors.
4. A comparative matrix is displayed to the user.

- **Alternative/Exception Flows:**

- *Missing Baselines*: The system discovers an empty control inventory and immediately halts, prompting the user to upload baseline configurations first.
- **Postconditions**: A security gap report is linked to the CVE profile.
- **Related Feature(s)**: F10.

UC-09: Filter Historical Repository

- **Use Case ID**: UC-09
- **Use Case Name**: Filter Historical Repository
- **Actor(s)**: Security Analyst
- **Goal**: Perform fast, LLM-free database queries to find specific advisories.
- **Preconditions**: Database is populated with vulnerabilities.
- **Trigger**: Analyst configures UI filter inputs and clicks "Search".
- **Main Success Scenario**:
 1. Analyst inputs query filters (e.g., CVSS > 7.0).
 2. The UI controller forms an SQL query via the repository layer.
 3. Matching database entities are retrieved and returned.
 4. The data table populates with the results.
- **Alternative/Exception Flows**:
 - *Invalid Thresholds*: The UI catches negative or out-of-bounds score inputs, immediately rejecting the query before execution.
- **Postconditions**: The user's screen reflects the filtered intelligence set.
- **Related Feature(s)**: F08.

Sequence Diagrams

1. SD-01: Natural Language Querying & Intelligence Retrieval

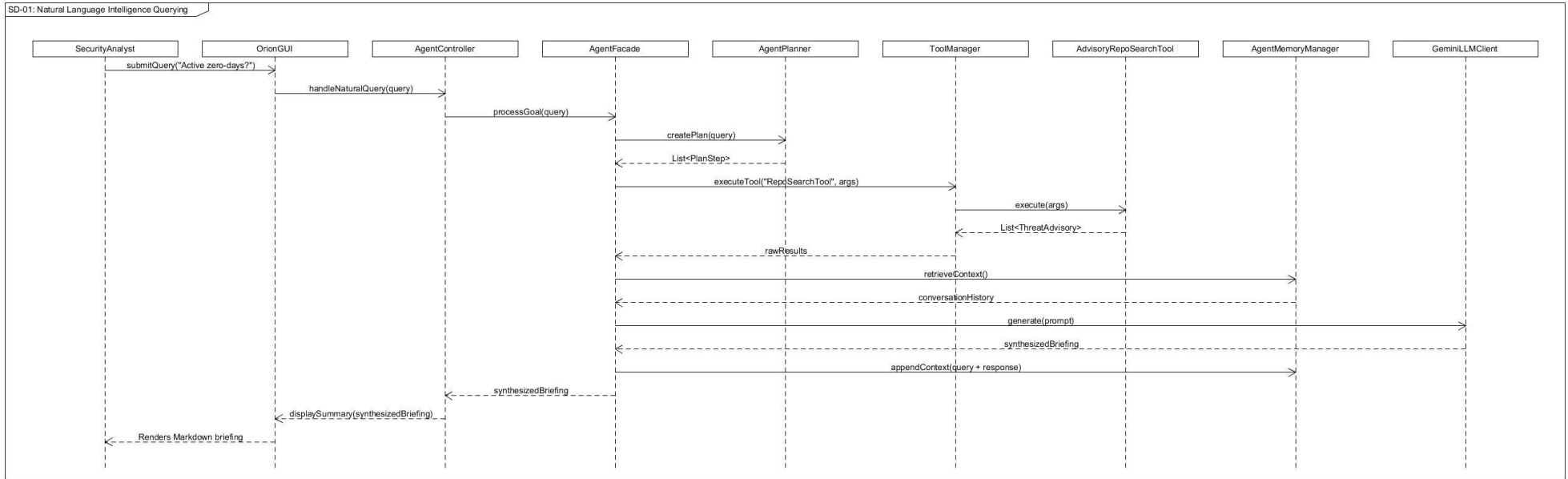


Figure 3: Sequence Diagram 1

- **Initiation:** The SecurityAnalyst inputs a natural language query into the OrionGUI.
- **Boundary to Controller:** OrionGUI invokes AgentController.handleNaturalQuery(query).
- **Facade Orchestration:** AgentController delegates execution to AgentFacade.processGoal(query).
- **Planning & Tool Execution:** AgentPlanner.createPlan() evaluates the goal, determining that a repository search is required. AgentFacade calls ToolManager.executeTool(), which invokes AdvisoryRepoSearchTool.execute() to pull matching data from the local database.
- **Memory & Prompt Framing:** AgentMemoryManager.retrieveContext() pulls recent conversational history. PromptBuilder formats this data alongside the retrieved records.

- **LLM Synthesis & Return:** GeminiLLMClient.generate() processes the prompt asynchronously. The resulting text is saved back to memory via AgentMemoryManager.appendContext() and returned through the facade and controller to OrionGUI.displaySummary().
- **Alternative Flow (Rate Limit):** If GeminiLLMClient encounters an HTTP 429 rate limit, the system catches the exception, bypasses the LLM call, and falls back to rendering raw database keyword matches.

2. SD-02: Insider Risk Log Correlation & Observer Notification

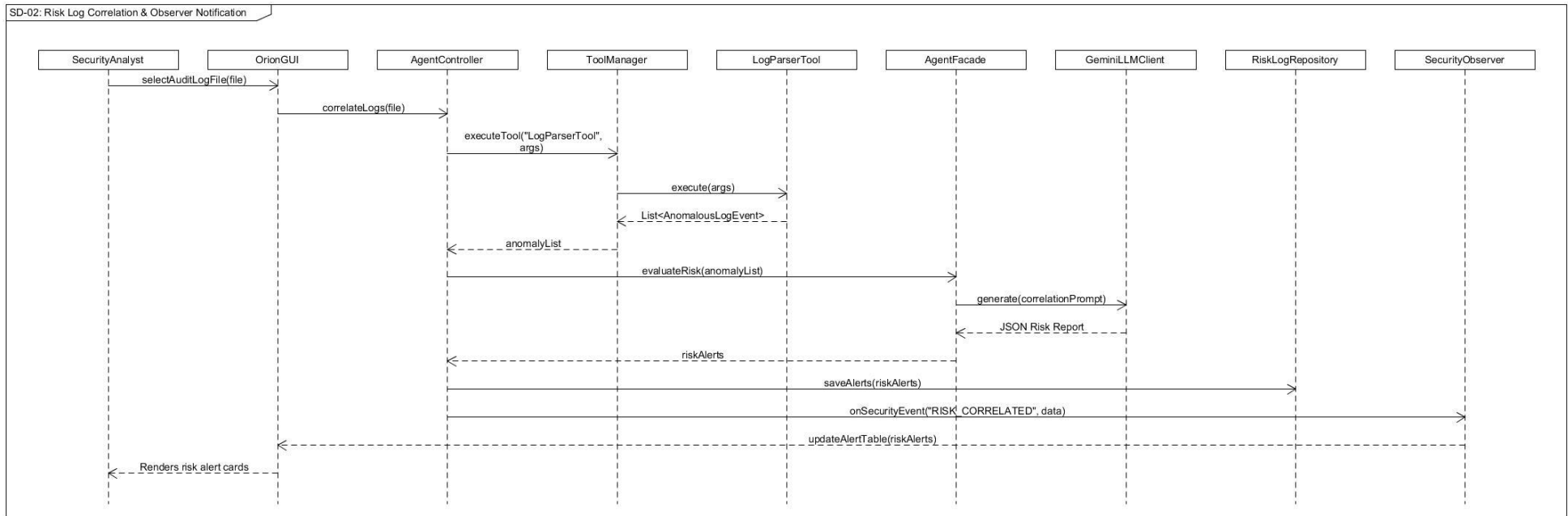


Figure 4: Sequence Diagram 2

- **Initiation:** The analyst uploads an internal audit log through OrionGUI.
- **Deterministic Parsing:** AgentController calls ToolManager.executeTool("LogParserTool", args), where LogParserTool.execute() deterministically parses the file and extracts abnormal access spikes.
- **AI Risk Evaluation:** AgentController passes the anomalies to AgentFacade.evaluateRisk(), which queries GeminiLLMClient.generate() to cross-reference the events against known threat signatures.

- **Persistence & Notification:** Structured RiskAlert records are saved via RiskLogRepository.saveAlerts().
- **Observer Broadcast:** To prevent UI blocking, AgentController notifies registered listeners by triggering SecurityObserver.onSecurityEvent(). OrionGUI (which implements the observer) catches this event and refreshes the risk table on the JavaFX application thread.
- **Alternative Flow (Malformed File):** If the CSV file contains invalid formatting, LogParserTool throws a LogValidationException, halting execution immediately and displaying an error popup without invoking external LLM tokens.

3. SD-03: Mitigation Plan Generation & Command Execution

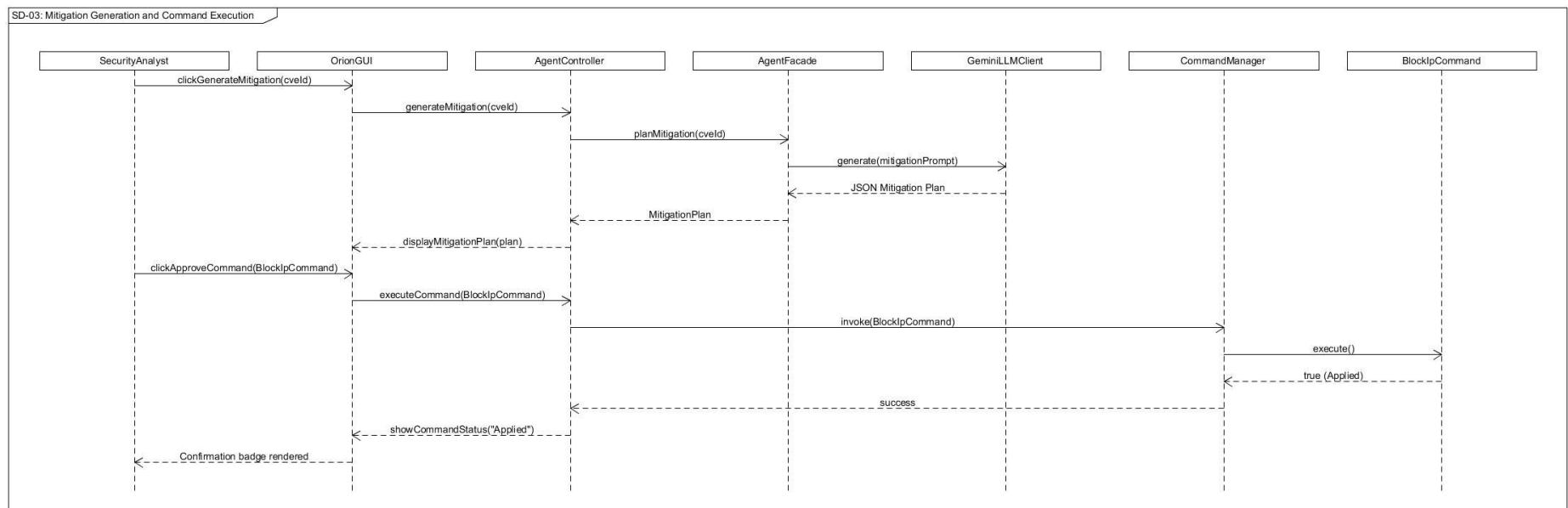


Figure 5: Sequence Diagram 3

- **Initiation:** The analyst clicks "Generate Mitigation" for a selected threat in OrionGUI.
- **Plan Synthesis:** AgentController calls AgentFacade.planMitigation(), which queries GeminiLLMClient using structured constraints to output a remediation playbook.

- **Command Object Instantiation:** The system parses the JSON response into a MitigationPlan containing concrete command instances, such as BlockIpCommand.
- **Human-in-the-Loop Validation:** The plan is displayed in the GUI. Upon analyst approval, OrionGUI triggers AgentController.executeCommand(cmd).
- **Command Invocation & Stacking:** AgentController hands the command to CommandManager.invoke(). The manager calls BlockIpCommand.execute() to apply the network rule and pushes the command onto its internal history stack.
- **Alternative Flow (Undo Operation):** If an administrator needs to revert the action, they click "Undo", which prompts CommandManager.undoLast() to pop the command and call BlockIpCommand.undo(), restoring the previous state.

Feature-to-Design Traceability

Feature	Description	Type	Related Use Case	Classes Involved	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Ingest and parse external threat feeds	Deterministic	UC-01: Ingest Threat Advisories	OrionGUI, AgentController, AdvisoryParserFactory, AdvisoryParser, NvdJsonParser, AdvisoryRepository	syncFeeds(), createParser(), parse(), saveAll()	SD-02 (Adapter/Flow)	Factory Method
F02	Automated vulnerability summarization	AI/Hybrid	UC-02: Query & Summarize Intel	OrionGUI, AgentController, AgentFacade, AgentPlanner, PromptBuilder, GeminiLLMClient	handleNaturalQuery(), processGoal(), buildQueryPrompt(), generate()	SD-01	Facade, Observer

F03	Natural language intelligence querying	AI/Hybrid	UC-02: Query & Summarize Intel	OrionGUI, AgentController, AgentFacade, AgentMemoryManager, GeminiLLMClient	handleNaturalQuery(), retrieveContext(), appendContext(), generate()	SD-01	Facade
F04	Correlate internal logs with external risk profiles	AI/Hybrid	UC-03: Correlate Insider Risk Logs	OrionGUI, AgentController, ToolManager, LogParserTool, AgentFacade, GeminiLLMClient	correlateLogs(), executeTool(), execute(), generate()	SD-02	Strategy, Observer
F05	Formulate and execute mitigation playbooks	AI/Hybrid	UC-04, UC-07: Mitigation & Execution	OrionGUI, AgentController, AgentFacade, GeminiLLMClient, CommandManager, BlockIpCommand	generateMitigation(), planMitigation(), generate(), invoke(), execute()	SD-03	Command, Facade
F06	Extract structured IoCs from unstructured text	AI/Hybrid	UC-05: Extract Structured IoCs	OrionGUI, AgentController, AgentFacade, AgentPlanner, GeminiLLMClient	handleNaturalQuery(), processGoal(), createPlan(), generate()	SD-01	Facade
F07	Export structured threat briefings	Deterministic	UC-06: Export Threat Briefing	OrionGUI, AgentController, AdvisoryRepository	searchAdvisories(), findById()	SD-01 (Query Flow)	Repository (Standard)

F08	Filter historical advisory repository	Deterministic	UC-09: Filter Historical Repository	OrionGUI, AgentController, AdvisoryRepository	searchAdvisories(), findByCriteria()	SD-01 (Query Flow)	Repository (Standard)
F09	Automated vulnerability triage and scoring	AI/Hybrid	UC-04: Formulate Mitigation & Triage	OrionGUI, AgentController, AgentFacade, GeminiLLMClient	generateMitigation(), evaluateTriage(), generate()	SD-03	Facade
F10	Evaluate enterprise security control gaps	AI/Hybrid	UC-08: Evaluate Control Gaps	OrionGUI, AgentController, AgentFacade, GeminiLLMClient	handleNaturalQuery(), evaluateGaps(), generate()	SD-01	Facade

Task to Feature Implementation Explanations

F01 — Ingest Threat Advisories

- **Related Use Case:** UC-01: Ingest Threat Advisories
- **Related Sequence Diagram:** SD-02 (Shared background sync pattern)
- **Classes involved:**
 - OrionGUI — captures the user's sync request.
 - AgentController — orchestrates the feed retrieval and storage.
 - AdvisoryParserFactory — creates the appropriate schema parser based on the feed type.
 - NvdJsonParser — parses the raw JSON vulnerability data.

- AdvisoryRepository — handles database transactions.
- **Important methods:**
 - OrionGUI.onSecurityEvent()
 - AgentController.syncFeeds()
 - AdvisoryParserFactory.createParser()
 - NvdJsonParser.parse()
 - AdvisoryRepository.saveAll()
- **Execution:** When the user initiates a sync in the GUI, syncFeeds() is called on the AgentController. The controller requests a parser from AdvisoryParserFactory, which returns an instance like NvdJsonParser. The parser transforms the raw HTTP response into domain objects. The controller then passes these objects to AdvisoryRepository.saveAll() for persistence.

F02 — Automated Vulnerability Summarization

- **Related Use Case:** UC-02: Query & Summarize Intel
- **Related Sequence Diagram:** SD-01: Natural Language Intelligence Querying
- **Classes involved:**
 - OrionGUI — displays the generated summary.
 - AgentController — routes the summarization request.
 - AgentFacade — manages the AI orchestration.
 - PromptBuilder — formats the threat data into an LLM constraint template.
 - GeminiLLMClient — interfaces with the external AI model.
- **Important methods:**

- AgentController.handleNaturalQuery()
- AgentFacade.processGoal()
- PromptBuilder.buildQueryPrompt()
- GeminiLLMClient.generate()
- **Execution:** The user selects an advisory and requests a summary. The GUI passes this to AgentController, which delegates to AgentFacade. The facade utilizes PromptBuilder to combine the raw vulnerability metrics with strict summarization constraints. This prompt is sent via GeminiLLMClient.generate(), and the resulting text is returned to the GUI for display.

F03 — Natural Language Conversational Intelligence Querying

- **Related Use Case:** UC-02: Query & Summarize Intel
- **Related Sequence Diagram:** SD-01: Natural Language Intelligence Querying
- **Classes involved:**
 - OrionGUI — accepts the chat input.
 - AgentController — acts as the deterministic router.
 - AgentFacade — coordinates the contextual AI components.
 - AgentMemoryManager — provides and stores conversation history.
 - GeminiLLMClient — generates the conversational response.
- **Important methods:**
 - AgentController.handleNaturalQuery()
 - AgentFacade.processGoal()
 - AgentMemoryManager.retrieveContext()

- AgentMemoryManager.appendContext()
- GeminiLLMClient.generate()
- **Execution:** The user enters a question. AgentController calls AgentFacade.processGoal(). The facade retrieves previous chat turns via AgentMemoryManager.retrieveContext() to maintain state. The prompt and memory are sent to GeminiLLMClient. The LLM's response is passed to AgentMemoryManager.appendContext() to update the memory stack before being surfaced in the GUI.

F04 — Correlate Internal Logs with External Risk Profiles

- **Related Use Case:** UC-03: Correlate Insider Risk Logs
- **Related Sequence Diagram:** SD-02: Risk Log Correlation & Observer Notification
- **Classes involved:**
 - OrionGUI — handles file upload.
 - AgentController — directs the parsing and AI evaluation workflow.
 - ToolManager — selects the correct parsing strategy.
 - LogParserTool — processes the raw CSV/syslog file.
 - AgentFacade — bridges the parsed data with the LLM.
 - GeminiLLMClient — identifies behavioral threat overlaps.
- **Important methods:**
 - AgentController.correlateLogs()
 - ToolManager.executeTool()
 - LogParserTool.execute()
 - AgentFacade.evaluateTriage()

- GeminiLLMClient.generate()
- **Execution:** The user uploads a log file. AgentController uses ToolManager to invoke LogParserTool.execute(), extracting structural anomalies deterministically. The controller passes these anomalies to AgentFacade, which queries GeminiLLMClient to evaluate the data against known external threat vectors. The correlated risk alerts are saved and the GUI is updated.

F05 — Formulate and Execute Mitigation Playbooks

- **Related Use Case:** UC-04: Formulate Mitigation & Triage, UC-07: Execute Security Command
- **Related Sequence Diagram:** SD-03: Mitigation Generation and Command Execution
- **Classes involved:**
 - OrionGUI — displays the plan and captures approval.
 - AgentController — coordinates generation and execution phases.
 - AgentFacade — requests the playbook from the AI.
 - CommandManager — queues and tracks executed system changes.
 - BlockIpCommand — encapsulates the specific network rule modification.
- **Important methods:**
 - AgentController.generateMitigation()
 - AgentController.executeCommand()
 - CommandManager.invoke()
 - BlockIpCommand.execute()
- **Execution:** The AI generates a MitigationPlan via the facade. When the user approves an action, the GUI calls executeCommand() with a specific implementation like BlockIpCommand. The controller hands the command to CommandManager, which calls execute() on the command object and pushes it onto the history stack for potential undo operations.

F06 — Extract Structured IoCs from Unstructured Text

- **Related Use Case:** UC-05: Extract Structured IoCs
- **Related Sequence Diagram:** SD-01 (Shared Query Flow)
- **Classes involved:**
 - OrionGUI — captures the raw text paste.
 - AgentController — passes the text to the intelligence subsystem.
 - AgentFacade — directs the extraction logic.
 - AgentPlanner — determines the required extraction steps.
 - GeminiLLMClient — de-fangs and normalizes the indicators.
- **Important methods:**
 - AgentController.handleNaturalQuery()
 - AgentFacade.processGoal()
 - AgentPlanner.createPlan()
 - GeminiLLMClient.generate()
- **Execution:** Unstructured text is submitted via OrionGUI. AgentController forwards it to AgentFacade. The planner identifies the need for Indicator of Compromise (IoC) extraction. The text is formatted and sent to GeminiLLMClient with a strict JSON-schema prompt to normalize IPs and domains. The facade parses the LLM output into structured data and returns it to the interface.

F07 — Export Structured Threat Briefings

- **Related Use Case:** UC-06: Export Threat Briefing
- **Related Sequence Diagram:** SD-01 (Shared determinist flow)

- **Classes involved:**

- OrionGUI — initiates the file save dialog.
- AgentController — orchestrates the data retrieval and file writing.
- AdvisoryRepository — fetches the selected database records.

- **Important methods:**

- AgentController.searchAdvisories()
- AdvisoryRepository.findById()

- **Execution:** The user selects threats and clicks export. AgentController queries AdvisoryRepository using findById() to pull the full threat records. The controller maps these domain objects into a Markdown or PDF template and executes standard Java I/O operations to write the file securely to the local disk.

F08 — Filter Historical Advisory Repository

- **Related Use Case:** UC-09: Filter Historical Repository

- **Related Sequence Diagram:** SD-01 (Shared deterministic flow)

- **Classes involved:**

- OrionGUI — captures search parameters.
- AgentController — translates parameters into repository queries.
- AdvisoryRepository — executes the SQL filters.

- **Important methods:**

- AgentController.searchAdvisories()
- AdvisoryRepository.findByCriteria()

- **Execution:** The user applies a filter (e.g., CVSS > 8.0) in OrionGUI. The boundary layer calls `AgentController.searchAdvisories()`. The controller bypasses the AI facade entirely and calls `AdvisoryRepository.findByCriteria()`, which executes a prepared SQL statement against the local database and returns the deterministic result list to the GUI.

F09 — Automated Vulnerability Triage and Severity Scoring

- **Related Use Case:** UC-04: Formulate Mitigation & Triage
- **Related Sequence Diagram:** SD-03: Mitigation Generation and Command Execution
- **Classes involved:**
 - OrionGUI — renders the calculated triage priority.
 - AgentController — initiates the scoring request.
 - AgentFacade — manages the AI scoring logic.
 - GeminiLLMClient — evaluates the qualitative threat context.
- **Important methods:**
 - `AgentController.generateMitigation()`
 - `AgentFacade.evaluateTriage()`
 - `GeminiLLMClient.generate()`
- **Execution:** When a new vulnerability is ingested or manually selected, `AgentController` calls `AgentFacade.evaluateTriage()`. The facade instructs `GeminiLLMClient` to evaluate the threat's qualitative description against the specific enterprise environment profile, returning a customized severity score that overrides the generic base CVSS rating in the GUI.

F10 — Evaluate Enterprise Security Control Gaps

- **Related Use Case:** UC-08: Evaluate Control Gaps
- **Related Sequence Diagram:** SD-01 (Shared Query Flow)

- **Classes involved:**

- OrionGUI — triggers the gap analysis.
- AgentController — manages the data collection.
- AgentFacade — bridges the baseline configurations and the LLM.
- GeminiLLMClient — compares controls against the threat vector.

- **Important methods:**

- AgentController.handleNaturalQuery()
- AgentFacade.evaluateGaps()
- GeminiLLMClient.generate()

- **Execution:** The user requests a gap analysis on a specific CVE. AgentController retrieves the enterprise's configured baseline security controls and passes them alongside the threat profile to AgentFacade.evaluateGaps(). The GeminiLLMClient performs a cross-reference matrix analysis to highlight missing defensive coverage, returning a gap report to the presentation layer.